Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий

Кафедра «Информационная безопасность»

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №2

по дисциплине
Информационная безопасность и защита информации

RSA
Вариант 13

Преподаватель			А. К. Зырянова
		подпись, дата	инициалы, фамилия
Студент	КИ24-07Б, 03254438		А.Д. Шахов
	номер группы, зачетной книжки	подпись, дата	инициалы, фамилия

Красноярск 2026

Задания к практической работе:

Согласно персональному варианту составить в виде блок-схемы алгоритм шифрования/расшифрования RSA, со следующими особенностями:

- объём исходного текста – любой (в разумных пределах);
- исходный текст может состоять из русских и английских букв, цифр, а также знаков препинания;
- исходный текст находится в кодировке ASCII;
- выступающее в качестве модуля число – N, которое выбирается автоматически и состоит из количества ДЕСЯТИЧНЫХ знаков, указанных в варианте
- Числа P и Q выбираются случайным образом, так, что P * Q = N , где P и Q – простые числа.
- исходный текст разбивается на K блоков, где K выбирается исходя из значения модуля N

Убедиться в правильности составления алгоритмов, а затем составить программу, которая реализует данный алгоритм.

На ряде контрольных примеров (не менее 5) открытого текста проверить правильность работы алгоритмов шифрования и расшифрования (в качестве контрольного примера понимается текстовый файл в кодировке ASCII).

Оценить криптостойкость варианта алгоритма RSA, а также сделать оценку производительности разработанной программы.

Персональный вариант представлен в таблице 1.

Таблица 1 – персональный вариант для выполнения работы.

№ варианта	**Шифр**	**N, количество знаков**
13	RSA	28

Подробное описание алгоритма шифрования:

Структура:

1. Генерация ключей:

- Выбираем два случайных простых числа p и q.
- Вычисляем n = p * q.
- Вычисляем phi = (p-1)*(q-1) (функция Эйлера).
- Выбираем e (открытый ключ) так, чтобы 1 < e < phi и gcd(e, phi) = 1.
- Находим d (закрытый ключ), обратное к e по модулю phi.

2. Шифрование текста:

- Текст переводится в массив байт (UTF-8 / ASCII).
- Делим текст на блоки размером blockSize = $\log_{256}(N)$.
- Каждый блок превращается в число.
- Шифруем: c = m^e mod n.

3. Расшифровка текста:

- Берём каждый зашифрованный блок c.
- Расшифровываем: m = c^d mod n.
- Превращаем число обратно в байты и склеиваем блоки.

Режимы работы:

Блочный режим (каждый блок обрабатывается отдельно)

UTF-8 / ASCII → поддержка русских и английских букв

Криптостойкость:

RSA считается безопасным, если числа p и q достаточно большие.

Вариант с N = 28 знаков (~93 бита) не очень надёжен по современным

меркам, в современном мире рекомендуется использовать не меньше 800 бит, но от размера числа снижается скорость работы алгоритма.

Без знания d расшифровать текст практически невозможно, только занимаясь факторизацией большого числа N.

Блок-схемы алгоритмов:

На рисунке 1 представлена блок-схема алгоритмов шифрования и расшифрования.

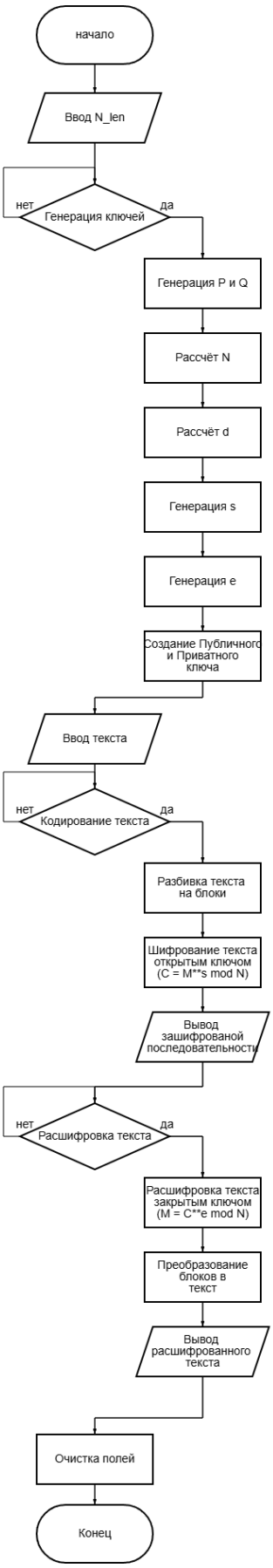

Рисунок 1 – Блок-схема алгоритма шифрования и расшифрования

Листинг программы:

```
import tkinter as tk
from tkinter import scrolledtext, messagebox
import random
import math


# проверка числа на простоту
def is_prime(n):
    if n <= 1:
        return False
    # Проверяем делители до корня из n
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False # Найден делитель, число составное
    return True # Делителей не найдено, число простое

# генерация рандомного простого числа с заданным количеством
цифр
def generate_prime(digits):
    while True:
        num = random.randrange(10**(digits-1), 10**digits)
        if is_prime(num):
            return num

#расширенный алгоритм евклида для вычисления закрытого ключа e
# a - d
# b - s (открытый ключ)
# алгоритм взят из статьи на хабре
# https://habr.com/ru/articles/745820
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0  # НОД, x, y
    else:
        d, x, y = extended_gcd(b, a % b)
        return d, y, x - y * (a // b)

#получение закрытого ключа
def mod_inverse(s, d):
    g, x, _ = extended_gcd(s, d)
    if g != 1:
        raise Exception("Закрытого ключа не существует")
    return x % d

# функция генерации ключей по заданной длине числа N
def generate_keys(n_digits):

    # получаем половину длины от нашей длины что бы
сгенерировать P и Q примерно равные по длине
    half = n_digits // 2
```

```
        # 1. P и Q
        # Генерируем P и Q, такие что они являются простыми и
равны половине знаков от N
        # Так же они не должны быть равными

        P = generate_prime(half)
        Q = generate_prime(n_digits - half)
        while P == Q:
            Q = generate_prime(n_digits - half)

        # 2. N
        # Вычисляем N

        N = P * Q

        # 3. d = фи(N)
        # Вычисляем d через формулу Элейра

        d = (P - 1) * (Q - 1)

        # 4. случайное s < d и gcd(s,d)=1
        # Получаем случайное число которое меньше чем d
        # у которого НОД с числом d равен 1

        while True:
            s = random.randrange(2, d)
            if math.gcd(s, d) == 1:
                break

        # 5. e*s ≡ 1 (mod d)
        # Вычисляем e по формуле из методы, используя расширенный
алгоритм евклида

        e = mod_inverse(s, d)

        # список публичного ключа и число модуля
        public_key = (s, N)
        # список приватного ключа и число модуля
        private_key = (e, N)

        return public_key, private_key, P, Q, d

    # вычисляем размерность блока
    def block_size(N):
        return max(1, int(math.log(N, 256))) #смотрим сколько байт
влезает в один блок

    # переводим текст в блоки
    def text_to_blocks(text, N):
        data = text.encode("utf-8") #преобразует текст в байты
        size = block_size(N)
```

```
        blocks = []
        for i in range(0, len(data), size):
            chunk = data[i:i+size] #берём срез данного размера
            blocks.append(int.from_bytes(chunk, "big"))
#преобразование байт в число

        return blocks


    def blocks_to_text(blocks, N):
        size = block_size(N)
        data = b''

        for b in blocks:
            data += b.to_bytes(size, "big").lstrip(b'\x00') #
lstrip убирает все 0 байты в начале, в случае если длина меньше
нашего размера блока

        return data.decode("utf-8") #преобразует байты в текст

    # шифрование при помощи открытого ключа
    def encrypt(blocks, public_key):
        encrypted_blocks = [] # здесь хранятся зашифрованные блоки

        for block in blocks:
            # Шифруем каждый блок по формуле C = M**s mod N
            encrypted_block = pow(block, public_key[0],
public_key[1]) # что, в какую степень, остаток от деления на
            encrypted_blocks.append(encrypted_block)

        return encrypted_blocks

    # расшифровка
    def decrypt(blocks, private_key):
        decrypted_blocks = []  # здесь хранятся расшифрованные
блоки

        for block in blocks:
            # Расшифровываем блоки по формуле M = C**e mod N
            decrypted_block = pow(block, private_key[0],
private_key[1]) # что, в какую степень, остаток от деления на
            decrypted_blocks.append(decrypted_block)

        return decrypted_blocks

    # С богом рабочий графический интерфейс
    class RSAApp:
        def __init__(self, root):
            self.root = root
            root.title("RSA GUI")

            self.public_key = None
```

```
            self.private_key = None

            # Метки
            tk.Label(root, text="Количество цифр N:").grid(row=0,
column=0, sticky="w")
            tk.Label(root, text="Исходный текст:").grid(row=1,
column=0, sticky="w")
            tk.Label(root, text="Закодированный
текст:").grid(row=3, column=0, sticky="w")
            tk.Label(root, text="Раскодированный
текст:").grid(row=5, column=0, sticky="w")
            tk.Label(root, text="Сгенерированные
ключи:").grid(row=7, column=0, sticky="w")

            # Поле ввода длины N
            self.n_digits_entry = tk.Entry(root, width=10)
            self.n_digits_entry.grid(row=0, column=1, sticky="w")
            self.n_digits_entry.insert(0, "28")  # значение по
умолчанию

            # Поля ввода/вывода текста
            self.input_text = scrolledtext.ScrolledText(root,
width=80, height=10)
            self.input_text.grid(row=2, column=0, columnspan=4,
padx=5, pady=5)

            self.encrypted_text = scrolledtext.ScrolledText(root,
width=80, height=10)
            self.encrypted_text.grid(row=4, column=0,
columnspan=4, padx=5, pady=5)

            self.decrypted_text = scrolledtext.ScrolledText(root,
width=80, height=10)
            self.decrypted_text.grid(row=6, column=0,
columnspan=4, padx=5, pady=5)

            self.keys_text = scrolledtext.ScrolledText(root,
width=80, height=10)
            self.keys_text.grid(row=8, column=0, columnspan=4,
padx=5, pady=5)

            # Кнопки
            tk.Button(root, text="Сгенерировать ключи",
command=self.generate_keys).grid(row=9, column=0, pady=10)
            tk.Button(root, text="Закодировать текст",
command=self.encode_text).grid(row=9, column=1, pady=10)
            tk.Button(root, text="Декодировать текст",
command=self.decode_text).grid(row=9, column=2, pady=10)
            tk.Button(root, text="Сбросить поля",
command=self.clear_fields).grid(row=9, column=3, pady=10)

        def generate_keys(self):
```

```
            try:
                digits = int(self.n_digits_entry.get())
                if digits < 2:
                    raise ValueError("Количество цифр должно быть
≥ 2")
                self.public_key, self.private_key, P, Q, phi =
generate_keys(digits)
                self.keys_text.delete("1.0", tk.END)
                self.keys_text.insert(tk.END, f"N =
{self.public_key[1]}\n")
                self.keys_text.insert(tk.END, f"Открытый ключ (s)
= {self.public_key[0]}\n")
                self.keys_text.insert(tk.END, f"Закрытый ключ (e)
= {self.private_key[0]}\n")
                self.keys_text.insert(tk.END, f"P = {P}\n")
                self.keys_text.insert(tk.END, f"Q = {Q}\n")
                self.keys_text.insert(tk.END, f"Фи = {phi}\n")
            except Exception as ex:
                messagebox.showerror("Ошибка", str(ex))

        def encode_text(self):
            if not self.public_key:
                messagebox.showwarning("Предупреждение", "Сначала
сгенерируйте ключи!")
                return
            text = self.input_text.get("1.0", tk.END).strip()
            if not text:
                return
            blocks = text_to_blocks(text, self.public_key[1])
            encrypted = encrypt(blocks, self.public_key)
            self.encrypted_text.delete("1.0", tk.END)
            self.encrypted_text.insert(tk.END, " ".join(map(str,
encrypted)))

        def decode_text(self):
            if not self.private_key:
                messagebox.showwarning("Предупреждение", "Сначала
сгенерируйте ключи!")
                return
            encrypted_str = self.encrypted_text.get("1.0",
tk.END).strip()
            if not encrypted_str:
                return
            try:
                blocks = list(map(int, encrypted_str.split()))
                decrypted_blocks = decrypt(blocks,
self.private_key)
                decrypted_text = blocks_to_text(decrypted_blocks,
self.private_key[1])
                self.decrypted_text.delete("1.0", tk.END)
                self.decrypted_text.insert(tk.END, decrypted_text)
            except Exception as ex:
```

```
            messagebox.showerror("Ошибка", str(ex))

    def clear_fields(self):
        self.input_text.delete("1.0", tk.END)
        self.encrypted_text.delete("1.0", tk.END)
        self.decrypted_text.delete("1.0", tk.END)
        self.keys_text.delete("1.0", tk.END)


if __name__ == "__main__":
    root = tk.Tk()
    app = RSAApp(root)
    root.mainloop()
```

Контрольные примеры работы программы:

На рисунках 2, 3, 4, 5, 6 представлены контрольные примеры работы программы с различными исходными текстами и ключами.

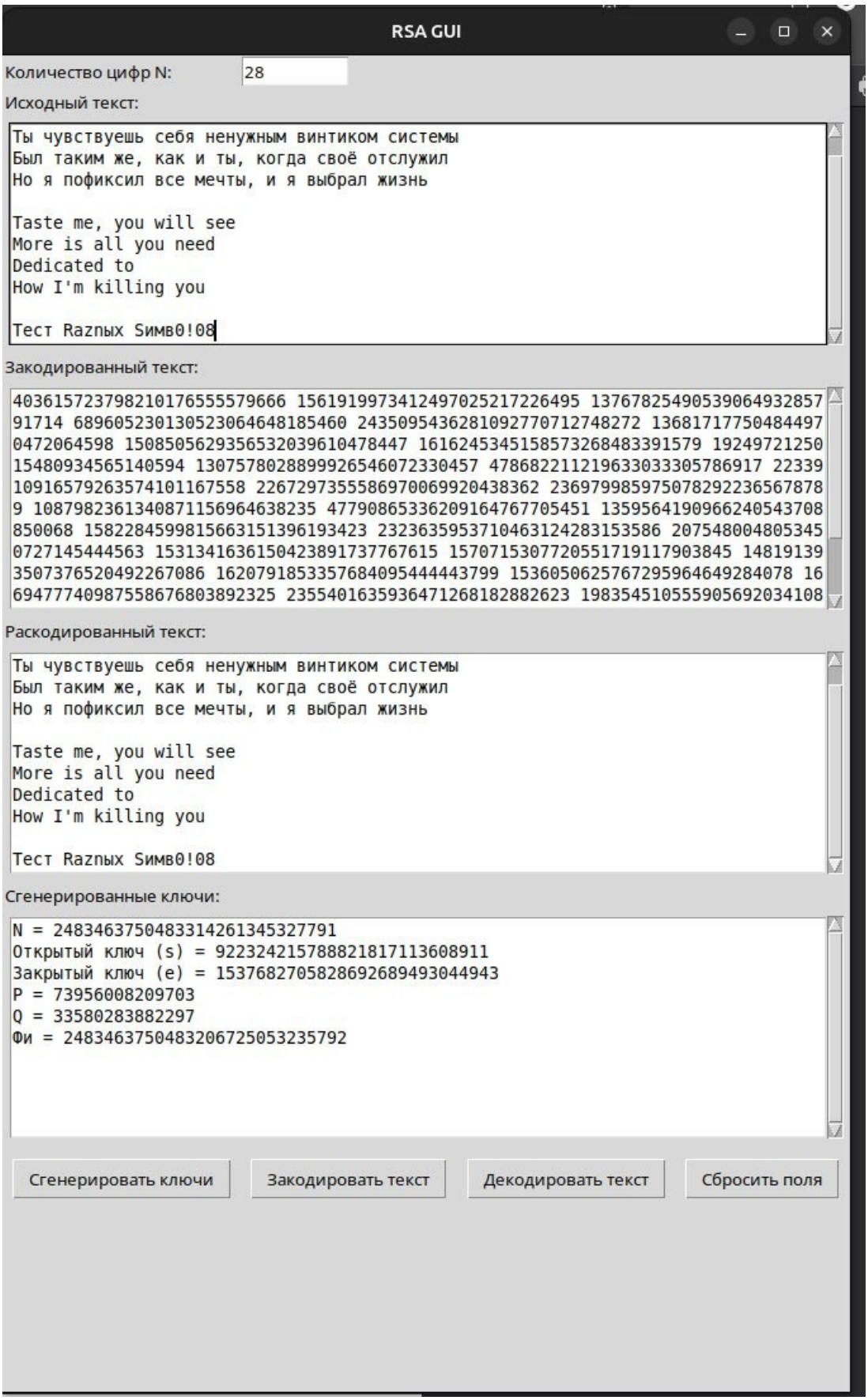

Рисунок 2 – Тест содержащий Кириллицу, Латиницу, спец символы

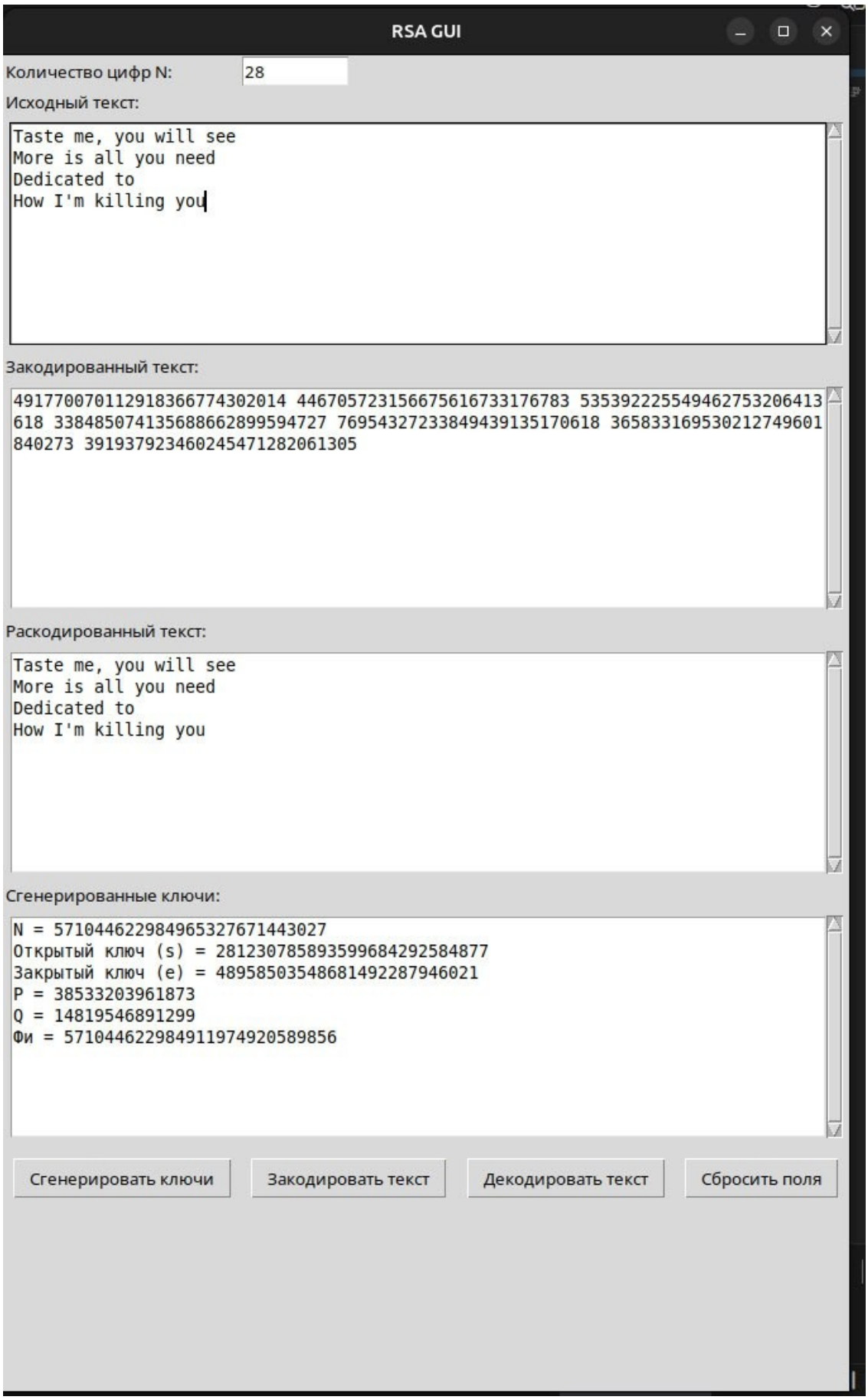

Рисунок 3 – Тест латинскими буквами

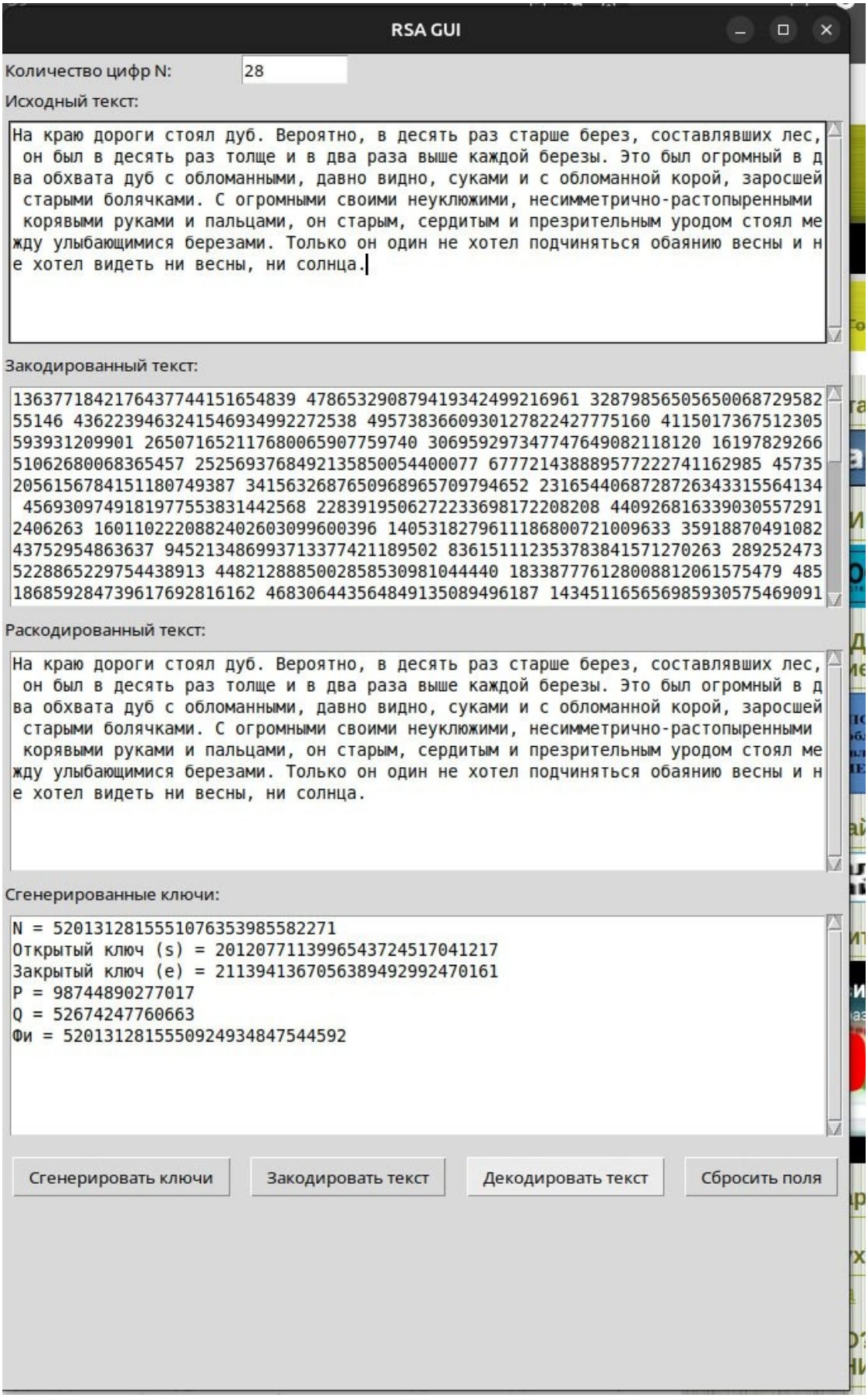

Рисунок 4 – Тест длинный текст

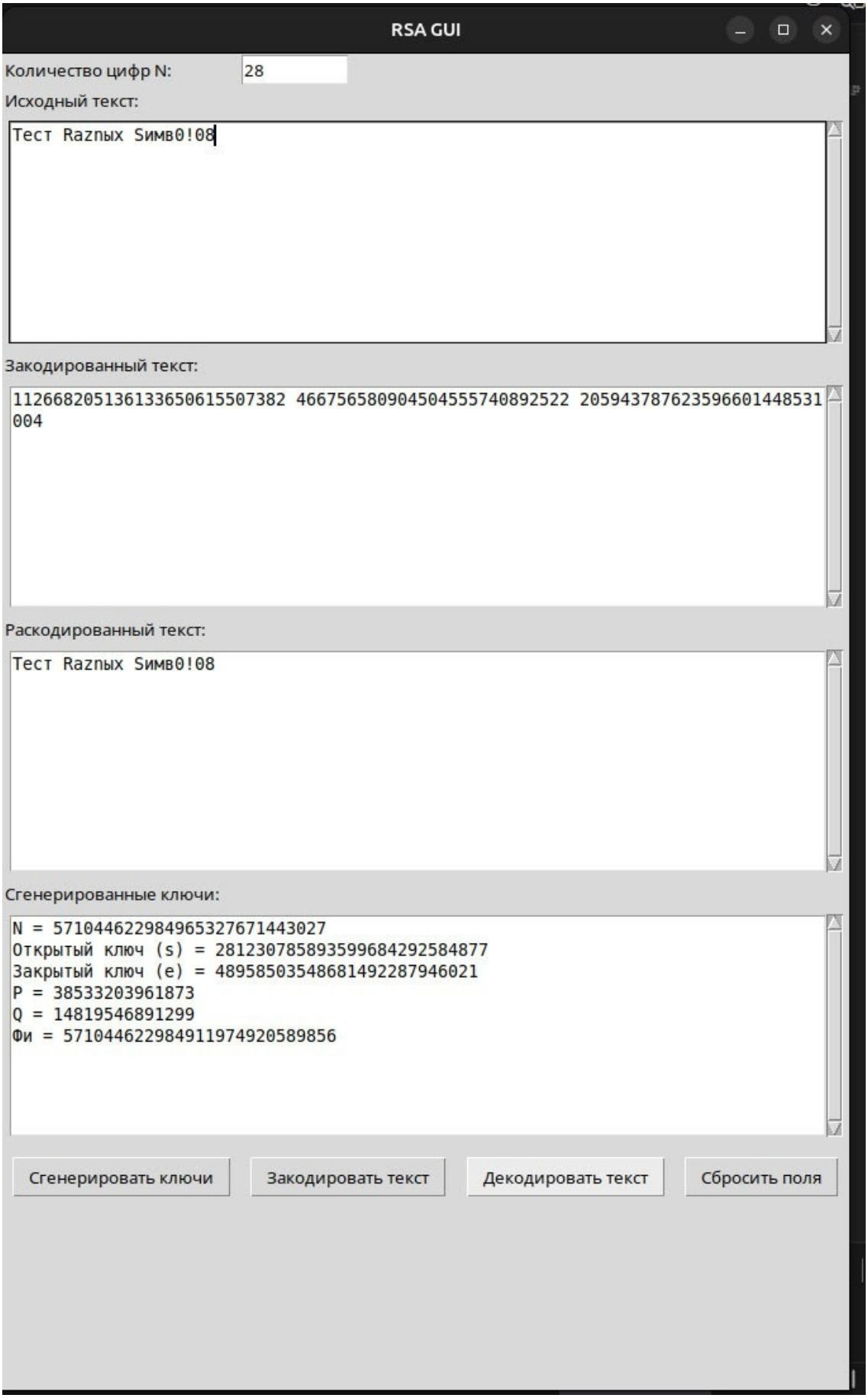

Рисунок 5 – Тест символами

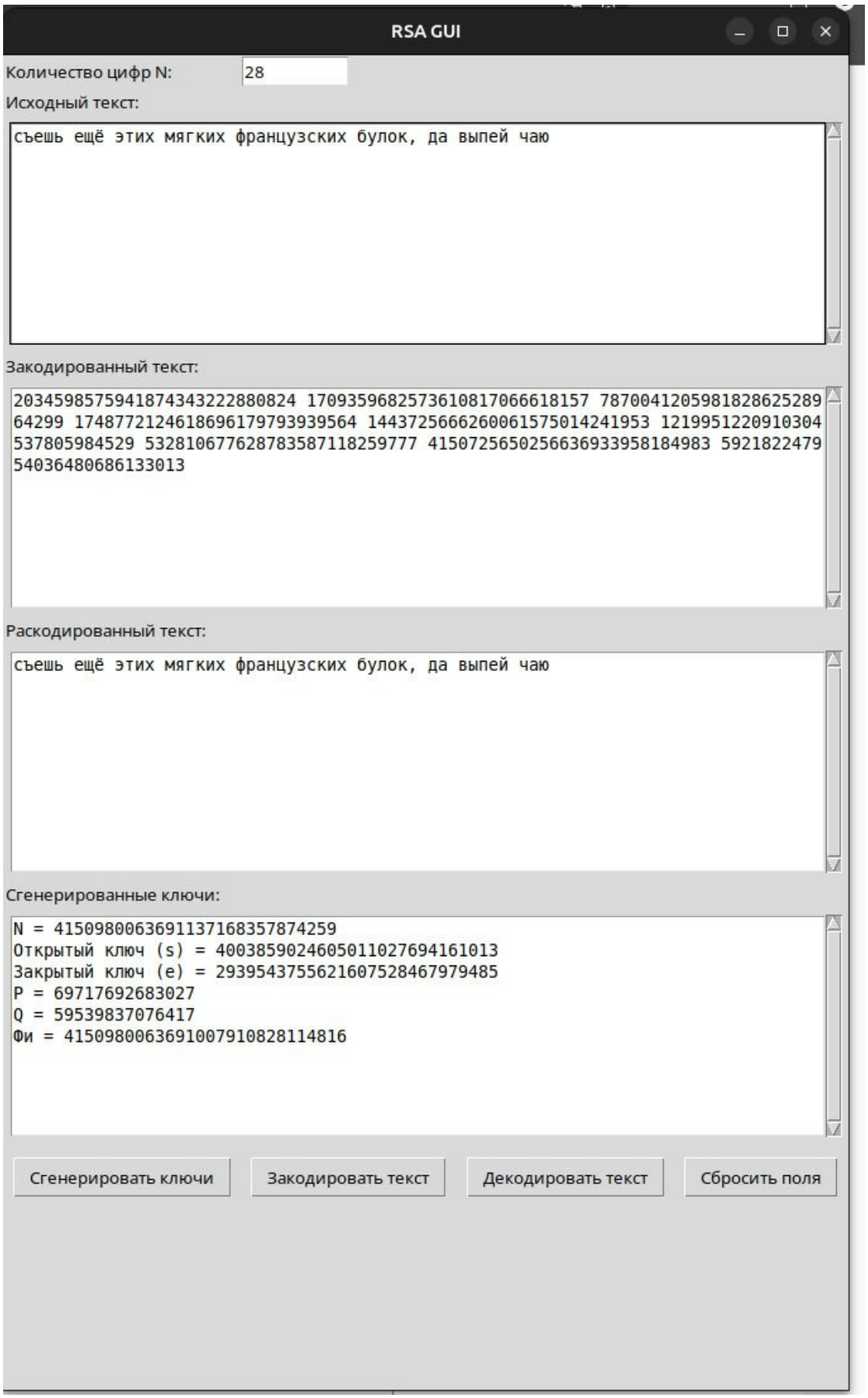

Рисунок 6 – Тест с другими ключами

Заключение: в ходе лабораторной работы был реализован алгоритм шифрования и расшифровки текста с использованием RSA для модуля N из 28 знаков. Алгоритм корректно работает с текстом на русском и английском языках, цифрами и знаками препинания в кодировке UTF-8. Криптостойкость алгоритма в рамках учебного варианта обеспечивает защиту. Программа показала адекватную производительность при обработке текста средней длины и корректно разделяет текст на блоки для шифрования. В ходе работы были закреплены навыки генерации ключей, работы с большими числами и преобразования текста в блоки, а также реализован графический интерфейс для взаимодействия с пользователем. Реализация показала принцип работы RSA, позволила понять основы асимметричной криптографии и получить практический опыт программирования на Python.